

HIGH LEVEL DESIGN (HLD)

Title

Design & Build a RAG-Based Customer Support Assistant

1. System Overview

This project implements a Retrieval-Augmented Generation (RAG) system for customer support.

Problem:

Traditional chatbots:

- Provide generic answers
- Do not access real documents

Solution:

A system that:

- Reads PDF knowledge base
- Retrieves relevant information
- Generates accurate responses
- Escalates complex queries

2. Architecture Diagram

System flow:

```
PDF --> Vector Embedding --> Vector DB
|
User --> Chat Interface --> Contextually Enhanced Query
^ |
| LLM API call
LLM Response <-----
```

3. Components Description

- User Interface -> Accepts user queries
- Document Loader -> Loads PDF
- Chunking Module -> Splits text into segments
- Embedding Model -> Converts text to vectors
- Vector Database (ChromaDB)-> Stores embeddings
- Retriever -> Fetches relevant chunks
- LLM -> Generates answers
- Workflow Engine (LangGraph)-> Controls processing flow

- Routing Layer -> Decides next action
- HITL Module -> Handles escalation to humans

4. Data Flow

- 1. Upload PDF
- 2. Extract text from PDF pages
- 3. Split text into chunks (size=300, overlap=50)
- 4. Generate embeddings using HuggingFace model
- 5. Store embeddings in ChromaDB
- 6. Receive user query
- 7. Retrieve top relevant chunks via similarity search
- 8. Generate answer from retrieved context
- 9. If low confidence -> escalate to HITL

5. Technology Choices

- ChromaDB -> Simple and efficient local vector database
- LangGraph -> Graph-based workflow control with state management
- Sentence Transformers -> Free, fast, high-quality embeddings (MiniLM-L6-v2)
- LangChain -> Standardised abstractions for loaders and chains
- Python -> Implementation language

6. Scalability

- Efficient chunking handles large documents
- Fast retrieval using vector similarity search
- Can handle multiple PDF documents in one collection
- LangGraph workflows are stateless per invocation - easy to parallelise